

MET CS688

WEB ANALYTICS AND MINING

ZLATKO VASILKOSKI

TERM PROJECTS

Outline of the CS688 Final Term Projects

- **Term Project (TP) Presentation will be held on the last day of classes. No presentations after that date.**
- **If needed, you can also present earlier (please arrange this early)**
- **The duration of each presentation should be at most 15 minutes.**

- **These slides contain term project suggestions and samples of student's TP from previous years.**
- **There is also a sample code that can be used as a starter code for many TP projects.**
  - **You can reuse that code or come up with your own.**

- **If applicable, integrate your code into a UI such as Shiny, Streamlit or Gradio App for higher grade.**
- **With your Python project Include "README.md" file that explains your code and libraries used, as illustrated in the sample Term Project Code.**
- **Try in your projects to use the latest technology which is about Local LLMs, RAG & AI Agents. This is what you will need to use if you find a job in the near future. The slides and the examples of how to use it are in the TP folder on Blackboard.**
- **It is very beneficial if you can use local or online AI service, integrate it in the form of an app and host it online. Hosting of AI Apps these days is very easy, is integrated with GitHub and is a routine part of your portfolio when you apply for jobs.**
  - **For AI (LLM) based apps you can do it for free at Huggingface Spaces.**
  - **For any other ML apps Streamlit is a great option.**

  At these sites you will find a very large number of apps that you can modify for the needs of your projects. Each modification of these published projects is automatically traced and properly credited.

TP should Include some (not all) of the following:

- Use of NLP Tasks on
  - Text from sources downloaded or scraped (TED talks, social media such as X, etc.).
  - Text from Images (using OCR, Visual Bert, Segment Anything, CLIP, etc.)
  - Speech converted to Text (your own audio files, downloaded, etc. apply NLP, back to speech).
  - Speech or Text from Video (web sites, YouTube etc.).
  - NLP on Text (such as summary of longer text) to Speech (audio)
- Use of ML
  - Using standard classification/regression ML algorithms on numeric data (financial or other)
  - Using reinforcement learning
  - Using standard classification/regression ML algorithms on text data (spaCy, Genism, NLTK, etc.)
  - Use of AutoML (Automated machine learning) (Google Cloud AutoML, H2O, Auto-SKLearn etc.)
- Use of latest ML technology (Transformers, BERT etc.)
  - Use of LLMs for standard NLP tasks
  - Use of Deep Learning for time series forecasting (financial data)
    - Use of Convolutional Neural Networks (CNNs)
    - Recurrent Neural Networks (RNNs)
    - Long Short-Term Memory (LSTM) RNNs
    - Google's Temporian
  - Integration of Image Processing with LLMs
- Use of latest AI
  - Generative AI Frameworks (built around LLMs) such as LangChain
  - Meta's FAISS, Llama, LlamaIndex
  - Google's tools such as PaLM 2

Some typical NLP Tasks include:

- Searching and Ranking
- Q&A
- Sentiment Analysis
- Keyword Extraction
- Text Summarization

Integrate your TP code with UI :

- As Strimlit App, Shiny App or similar.

Note: **That the use of latest tools and technology (and integration into UI) in your TP carries higher grade.**

Note: After choosing the type of your TP, besides the suggested tools, do an online search to explore what other tools you can use for your TP. These days the field of ML & NLP is constantly changing and every few months new and better tools come up.

In general, be inventive, come up with tasks that you find interesting and use this opportunity to get introduced to this latest NLP techniques and models which you will find very useful in your future career.

Typical Term Project Steps

Your TP should follow these general steps (as outlined and illustrated in the class slides):

1. Define the Problem.
   - What is the problem?
   - Why does the problem need to be solved?
   - How would I solve the problem?
2. Prepare the Data.
   - Data Selection – available, missing and what data can be removed.
   - Data Preprocessing – formatting, cleaning and sampling.
   - Data Transformation – feature engineering, scaling, attribute decomposition and attribute aggregation.
3. Evaluate the Algorithms.
   - Try few standard algorithms on the dataset to select the best algorithm.
4. Improve the Results.
   - Algorithm Tuning – select best parameters in model's parameter space.
   - Ensemble Methods – if needed, combine predictions from multiple models.
5. Present the Results. A formal report or presentation slides containing:
   - Context (Why) – set up the motivation for the work done.
   - Problem (Question) – concisely describe the problem as a question that needs to be answered.
   - Solution (Answer) – describe the solution as an answer to the posed problem.
   - Findings – relevant lists of discoveries (in data, methods, models, what didn't work) made in your work.
   - Limitations – describe questions (problems) the model cannot answer or does not perform well.
   - Conclusions – concise short, easy to remember summary of the motivation, the problem and the answer provided.

BERT Related Term Project Suggestions

You can use the python starter code on Blackboard which is illustrated on several NLP tasks

- You can use Python & BERT Models for **any** NLP tasks such as Q&A, Sentiment Analysis, Name entity recognition, keyword extraction, language generation etc.

You can also use any online resources such as Huggingface site:

https://huggingface.co/transformers/v2.5.1/examples.html

Section	Description
TensorFlow 2.0 models on GLUE	Examples running BERT TensorFlow 2.0 model on the GLUE tasks.
Language Model training	Fine-tuning (or training from scratch) the library models for language modeling on a text dataset. Causal language modeling for GPT/GPT-2, masked language modeling for BERT/RoBERTa.
Language Generation	Conditional text generation using the auto-regressive models of the library: GPT, GPT-2, Transformer-XL and XLNet.
GLUE	Examples running BERT/XLM/XLNet/RoBERTa on the 9 GLUE tasks. Examples feature distributed training as well as half-precision.
SQuAD	Using BERT/RoBERTa/XLNet/XLM for question answering, examples with distributed training.
Multiple Choice	Examples running BERT/XLNet/RoBERTa on the SWAG/RACE/ARC tasks.
Named Entity Recognition	Using BERT for Named Entity Recognition (NER) on the CoNLL 2003 dataset, examples with distributed training.
XNLI	Examples running BERT/XLM on the XNLI benchmark.
Adversarial evaluation of model performances	Testing a model with adversarial evaluation of natural language inference on the Heuristic Analysis for NLI Systems (HANS) dataset (McCoy et al., 2019.)

BERT Related Term Project Suggestions

An illustration of a BERT related Term Projects.

- **Question & Answering** (Q&A) system - Build Q&A, select a few pages of text to analyze and provide answers (most relevant sentences from the text) to the posed questions. For example,

```
question = "Who ruled ancient Macedonia?"

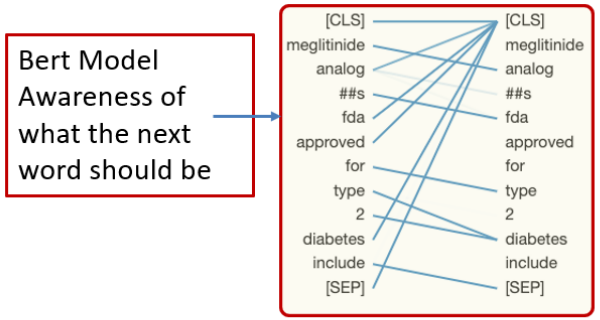
text = """Macedonia was an ancient kingdom on the periphery of Archaic and Classical Greece,
and later the dominant state of Hellenistic Greece. The kingdom was founded and initially ruled
by the Argead dynasty, followed by the Antipatrid and Antigonid dynasties. Home to the ancient
Macedonians, it originated on the northeastern part of the Greek peninsula. Before the 4th
century BC, it was a small kingdom outside of the area dominated by the city-states of Athens,
Sparta and Thebes, and briefly subordinate to Achaemenid Persia."""

# ==================
# Question: Who ruled ancient Macedonia?
# Answer: the Argead dynasty
# ==================
```

- Have a look at https://qa.fastforwardlabs.com/pytorch/hugging%20face/wikipedia/bert/transformers/2020/05/19/Getting_Started_with_QA.html or similar example online. Understand the code and build a Q&A System using any of the BERT models that will provide the answer from the Wikipedia content. For example:

  Question: “Why is the sky blue?”

  Answer pulled from Wikipedia content: “Rayleigh scattering”



For any of these BERT projects use visualization of BERT layers:
https://github.com/jessevig/bertviz

For visualization of BERT layers, use:
https://github.com/jessevig/bertviz

BERT Related Term Project Suggestions

Other examples:

- **Question Generation** - Explore the provided “Question Generation” code that generates questions from given text. Integrate this with and Q&A code and check if it returns the right answer.

  Example: for the following input text

  "Sachin Ramesh Tendulkar is a former international cricketer from India and a former captain of the Indian national team.”

  "He is widely regarded as one of the greatest batsmen in the history of cricket.”

  "He is the highest run scorer of all time in International cricket.”

  These are the questions that were generated:

  “What is Sachin Ramesh Tendulkar’s career?”

  “Where is Sachin Ramesh Tendulkar from?”

- **Speech to Text** - record yourself reading a page of text (TED Talk for example) as wav file and then use the provided “Wav2vec” code to turn it into a text that you can summarize using any NLP summary method. Estimate correctness of the voice to text transcription (how many corrections to the text you needed to make) and illustrate what was the summary of the initial text.

- **Keywords Extraction** - Use provided code to extract keywords from few pages of text (TED Talk for example). Then summarize the text by different methods and extract the keywords again. Compare if the quality of the extracted keywords improved.

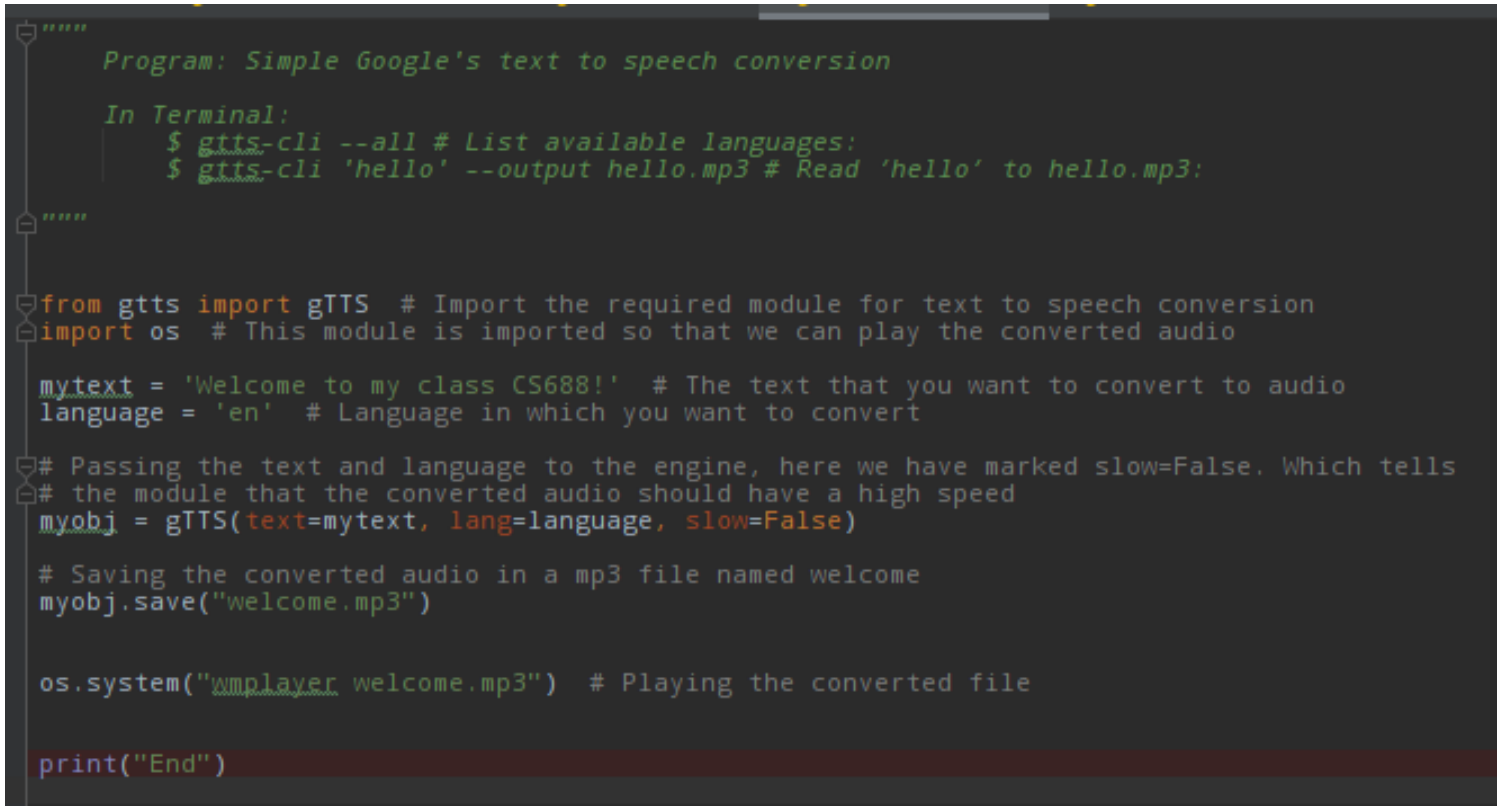
- Text sentiment, Text classification, Semantic Similarity, Next sentence prediction…

  Use visualization of BERT layers: https://github.com/jessevig/bertviz

BERT Related Term Project Suggestions

Other examples:

- **Text to Speech** - Convert text into natural-sounding speech using “gtts” library to interface with Google Translate's text-to-speech API.
- Links:
  - https://gtts.readthedocs.io/en/latest/cli.html#examples



```
"""
    Program: Simple Google's text to speech conversion

    In Terminal:
        $ gtts-cli --all # List available languages:
        $ gtts-cli 'hello' --output hello.mp3 # Read 'hello' to hello.mp3:

"""


from gtts import gTTS  # Import the required module for text to speech conversion
import os  # This module is imported so that we can play the converted audio

mytext = 'Welcome to my class CS688!'  # The text that you want to convert to audio
language = 'en'  # Language in which you want to convert

# Passing the text and language to the engine, here we have marked slow=False. Which tells
# the module that the converted audio should have a high speed
myobj = gTTS(text=mytext, lang=language, slow=False)

# Saving the converted audio in a mp3 file named welcome
myobj.save("welcome.mp3")


os.system("wmplayer welcome.mp3")  # Playing the converted file

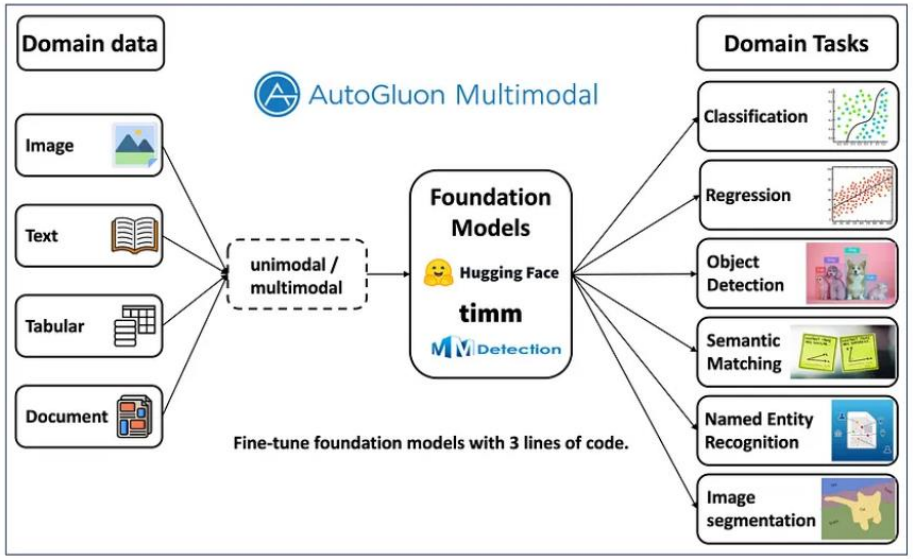

print("End")
```

Term Project Suggestions using latest technology

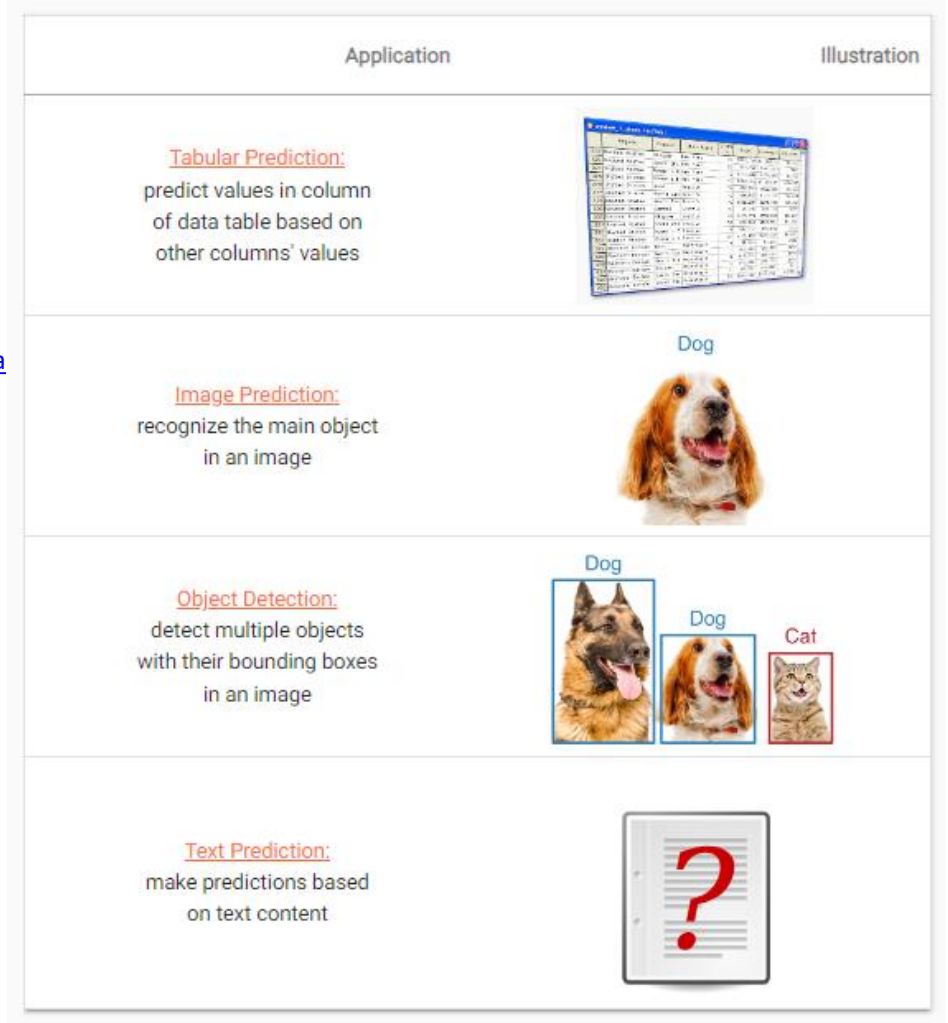
Amazon recently announced the launch of **AutoGluon**, a new **open-source** library for developers building applications involving machine learning with image, text, or tabular data sets. It enables easy-to-use and easy-to-extend **AutoML** with a focus on automated stack ensembling, deep learning, and real-world applications spanning text, image, and tabular data.

AutoGluon offers solutions for problems such as Image classification, Object detection,
Text prediction, Image segmentation, Time Series forecasting and much more

Link:
https://auto.gluon.ai/0.3.1/index.html#:~:text=AutoGluon%20enables%20easy%2Dto%2Duse,%2C%20image%2C%20and%20tabular%20data



Source: https://auto.gluon.ai/stable/tutorials/multimodal/index.html



TP Suggestions AutoML Example - AutoGluon

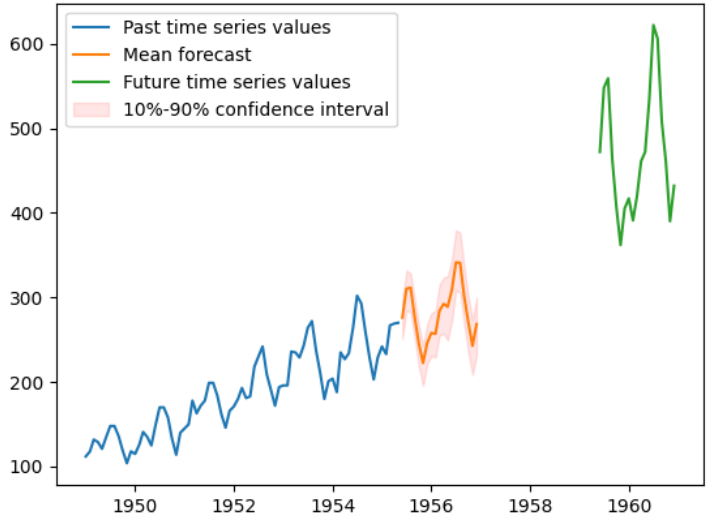
Example of Amazon's Multimodal AutoML library with a time series forecasting problem.

See Python project "AutoGluon_example" in Blackboard's sample code.

- Predictions on "airline-passengers"Dataset (included with the project).
- Train & Predict on Various Available Models:

  ['SeasonalNaive', 'CrostonSBA', 'NPTS', 'AutoETS', 'DynamicOptimizedTheta', 'AutoARIMA', 'RecursiveTabular', 'DirectTabular', 'DeepAR', 'TemporalFusionTransformer', 'PatchTST', 'WeightedEnsemble']
- If you don't specify prediction model, the best model will be considered.
- By default, AutoGluon predicts quantile levels [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9].

```
Best model score: -0.2104
Training Duration: 0:10:00.028250
                      model  score_val  ...  fit_time_marginal  fit_order
0          WeightedEnsemble  -0.210389  ...           4.370197         12
1          RecursiveTabular  -0.556043  ...           5.139443          7
2                 AutoARIMA  -0.747620  ...           0.025250          6
3   TemporalFusionTransformer  -0.781797  ...         273.860254         10
4                   AutoETS  -0.815171  ...           0.028496          4
5             SeasonalNaive  -0.872826  ...           0.021104          1
6                    DeepAR  -0.918107  ...         104.713793          9
7                  PatchTST  -1.116327  ...          71.116385         11
8       DynamicOptimizedTheta  -1.446013  ...           0.036431          5
9                 CrostonSBA  -1.474456  ...           0.037980          2
10                     NPTS  -2.672510  ...           0.027722          3
11             DirectTabular  -3.558071  ...           2.308942          8
```

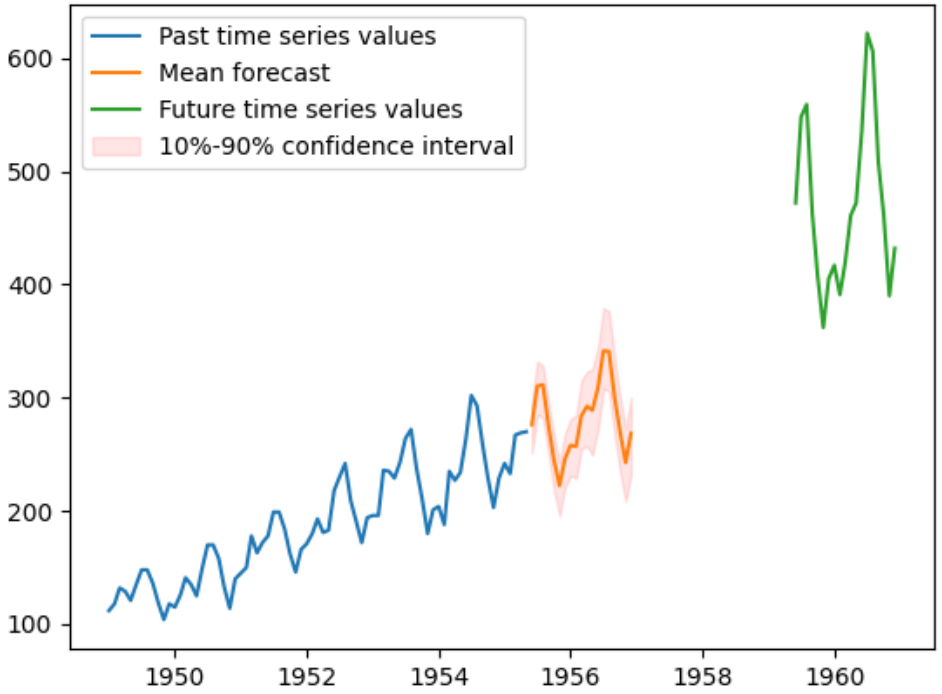


TP Suggestions AutoML Example - AutoGluon

Train & Predict on Specific Model:

- **DeepAR** – First Model that Combines Deep Learning with traditional Probabilistic Forecasting.

```
# Train using DeepAR
predictor = TimeSeriesPredictor(target='Passengers', prediction_length=19).fit(
    train,
    hyperparameters={
        "DeepAR": {},
    },
)
# Make Predictions using DeepAR
predictions = predictor.predict(train)
```



Term Project Suggestions - LangChain

LangChain provides a generic interface for many different LLMs.

Most of them work via their API, but you can also run local models.

- LangChain & OpenAI Example: Answer to a Question.

```python
from langchain.llms import OpenAI
import os


1 usage
def openai_langchain(text):
    with open("../../my_keys/my_openai_key.txt", mode='r', encoding='utf8') as f:
        my_key = f.readlines()[0]  # Read Saved OpenAI Key

    os.environ["OPENAI_API_KEY"] = my_key

    llm = OpenAI(temperature=0.9)  # model_name="text-davinci-003"
    reply = llm(text)
    return reply


question = "What would be a good name for a book on AI and comedy?"
answer = openai_langchain(question)

print("------------------------------------------------------------")
print("Question: " + question)
print(answer)
print("------------------------------------------------------------")
```

```
------------------------------------------------------------
Question: What would be a good name for a book on AI and comedy?


1. "Artificially Funny: How AI is Revolutionizing Comedy"
2. "Laughing with Machines: The Intersection of AI and Humor"
3. "From Algorithms to Laughter: Exploring the Funny Side of AI"
4. "Comedic Code: Unveiling the Humorous Potential of Artificial Intelligence"
5. "The Humor of Tomorrow: A Look at AI and the Future of Comedy"
6. "AI and the Art of Making People Laugh"
7. "Cracking the AI Code: The Science and Comedy of Artificial Intelligence"
8. "The Funny Side of Machines: A Journey into AI and Comedy"
9. "Laughing at Robots: The Evolution of AI in Comedy"
10. "Beyond the Punchline: How AI is Redefining Humor and Entertainment"
------------------------------------------------------------
```

```
------------------------------------------------------------
Question: What would be a good name for a book on AI and comedy?


"Artificial Laughter: Exploring the Relationship between AI and Humor"
------------------------------------------------------------
```

Term Project Suggestions - LangChain

LangChain & LLM Huggingface Example: Language Translation

```python
"""
    Program: Functions used in Project LangChain

"""

from langchain.llms import OpenAI
from langchain_community.llms import HuggingFaceEndpoint
import os


1 usage
def openai_langchain(text):
    with open("../../my_keys/my_openai_key.txt", mode='r', encoding='utf8') as f:
        my_key = f.readlines()[0]  # Read Saved OpenAI Key

    os.environ["OPENAI_API_KEY"] = my_key

    llm = OpenAI(temperature=0.9)  # model_name="text-davinci-003"
    reply = llm(text)
    return reply

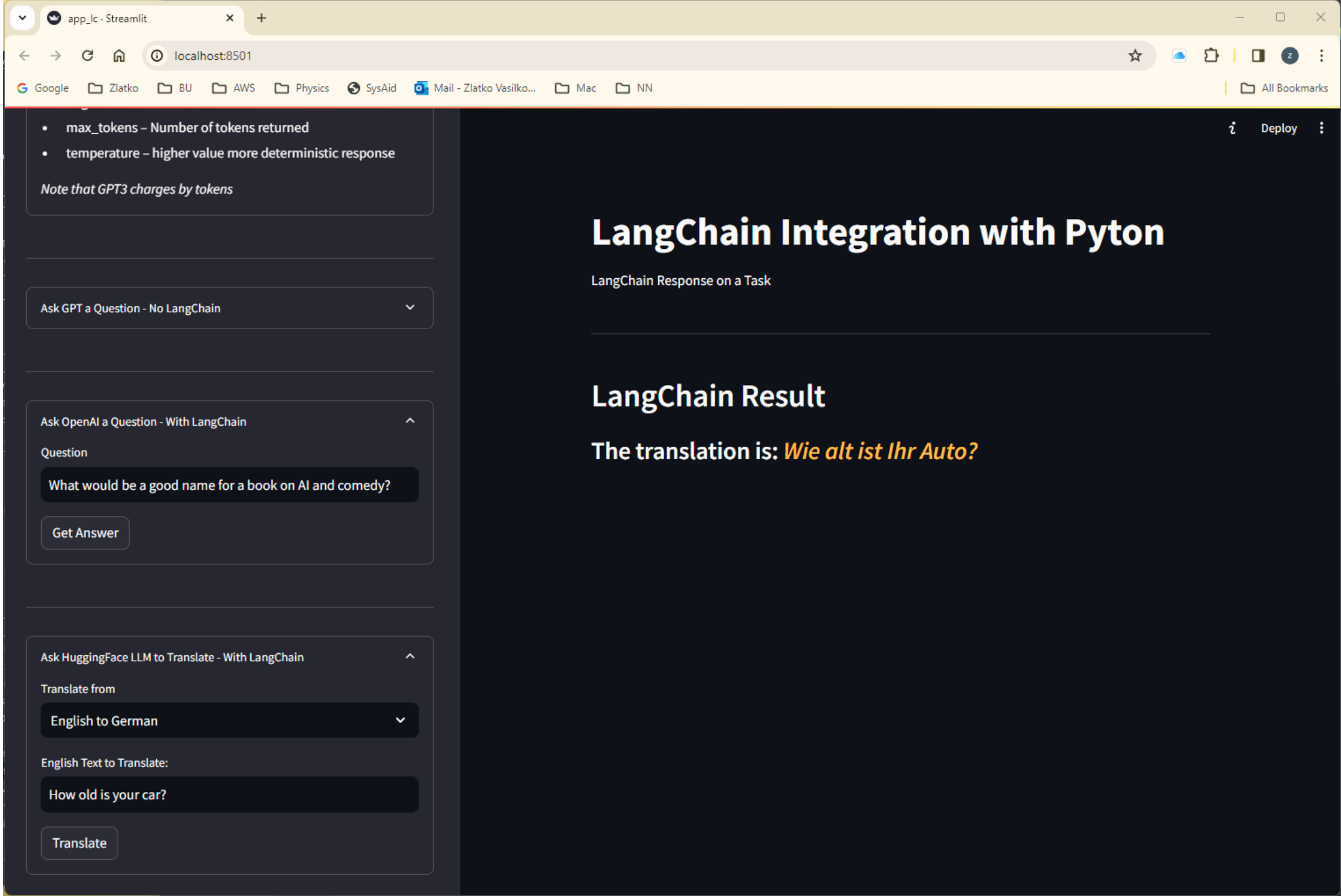

1 usage
def hf_langchain(text):
    with open("../../my_keys/my_huggingface_key.txt", mode='r', encoding='utf8') as f:
        my_token = f.readlines()[0]  # Read Saved Huggingface Key

    os.environ["HUGGINGFACEHUB_API_TOKEN"] = my_token

    # llm = HuggingFaceHub(repo_id="google/flan-t5-xxl", model_kwargs={"temperature": 0.8, "max_length": 512})
    llm = HuggingFaceEndpoint(repo_id="google/flan-t5-xxl", max_length=128, temperature=0.5, token=my_token)
    return llm
```

```
----------------------------------------------------
Translate English to French: How old is your car?
Translation:  quel an s'est ta voiture?
----------------------------------------------------
```

LangChain Streamlit App



Term Project Suggestions - LangChain

LangChain & LLM Huggingface Example: Prompts & Chains

Prompt Templates: LangChain facilitates prompt management and optimization.

- Normally, when you use an LLM in an application, you are not sending user input directly to the LLM. Instead, you need to take the user input and construct a prompt, and only then send that to the LLM.
- Example:

```
llm(" Can Gilgamesh be a character in a William Shakespeare's play?")
```

A better prompt is this:

```
prompt = \"""Question: Can Gilgamesh be a character in a William Shakespeare's play?
 Let's think step by step.
Answer: \"""
llm(prompt)
```

Chains: Combine LLMs and Prompts in multi-step workflows.

```
llm_chain = LLMChain(prompt=prompt, llm=llm)
question = "Can Gilgamesh be a character in a William Shakespeare's play?"
print(llm_chain.run(question))
```

William Shakespeare's plays are based on works of Greek mythology. Gilgamesh is a character in the ancient Sumerian epic poem The Epic of Gilgamesh. The Epic of Gilgamesh is set in ancient Babylon. The answer: no.

Term Project Suggestions using latest technology

Generative AI Frameworks (built around LLMs)

- LangChain (https://www.pinecone.io/learn/series/langchain/langchain-intro/)
  - Used for: chatbots, Generative Question-Answering (GQA), summarization, and much more.
  - Integrates with
    - Hugging Face Hub LLMs
    - OpenAI LLMs
    - Meta's Llama 2 LLM



LangChain AI Handbook

Chapters

1. An Introduction to LangChain
   - LangChain
   - Our First Prompt Templates
   - Hugging Face Hub LLM
   - OpenAI LLMs
   - References
2. Prompt Templates and the Art of Prompts
3. Building Composable Pipelines with Chains

Asking Multiple Questions

If we'd like to ask multiple questions, we can try two approaches:

1. Iterate through all questions using the `generate` method, answering them one at a time.
2. Place all questions into a single prompt for the LLM; this will only work for more advanced LLMs.

Starting with option (1), let's see how to use the `generate` method:

In[4]:

```
qs = [
    {'question': "Which NFL team won the Super Bowl in the 2010 season?"},
    {'question': "If I am 6 ft 4 inches, how tall am I in centimeters?"},
    {'question': "Who was the 12th person on the moon?"},
    {'question': "How many eyes does a blade of grass have?"}
]
res = llm_chain.generate(qs)
res
```

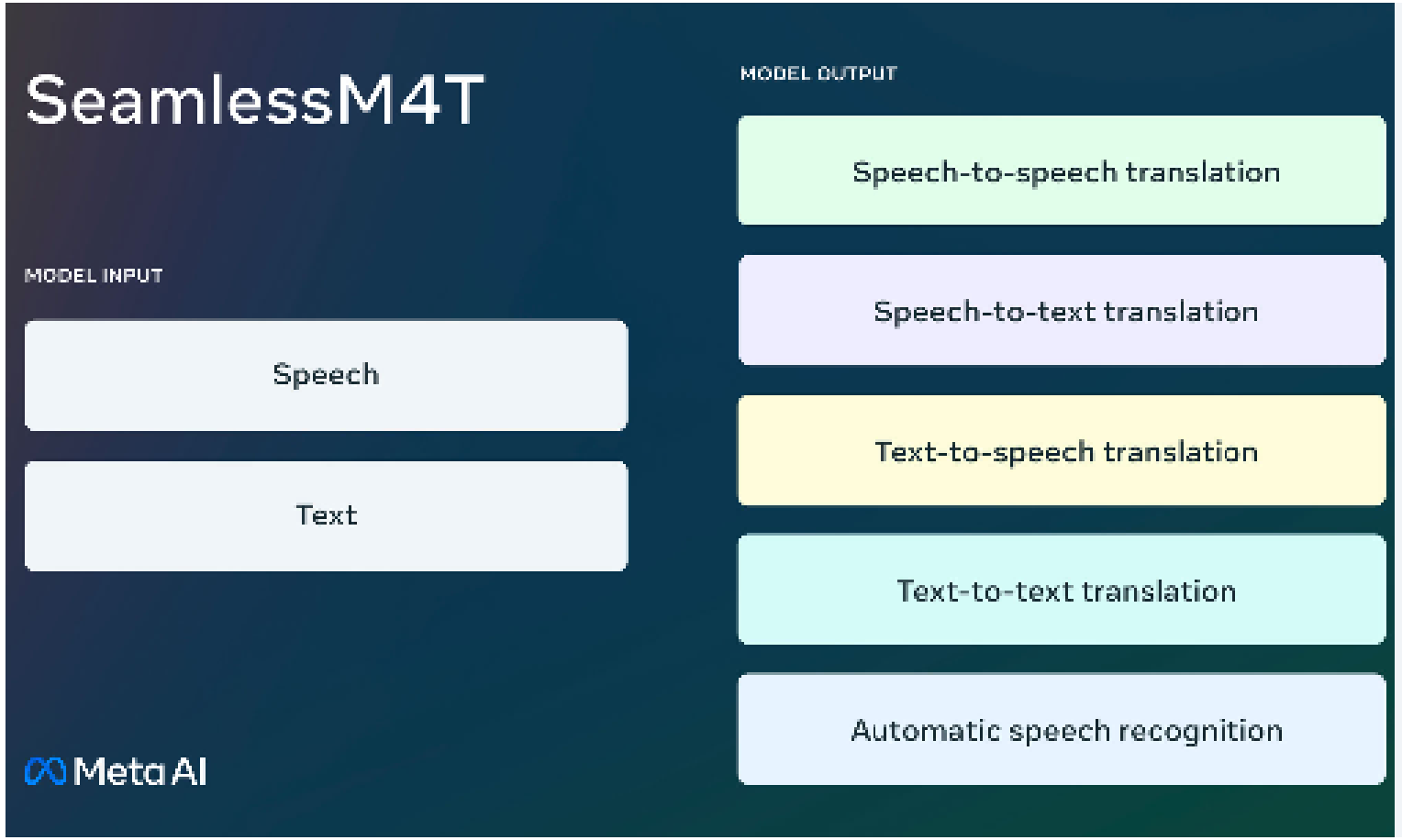
Out[4]:

```
LLMResult(generations=[[Generation(text='green bay packers', generation_info
```

Term Project Suggestions using latest technology

Meta's Tools such as:

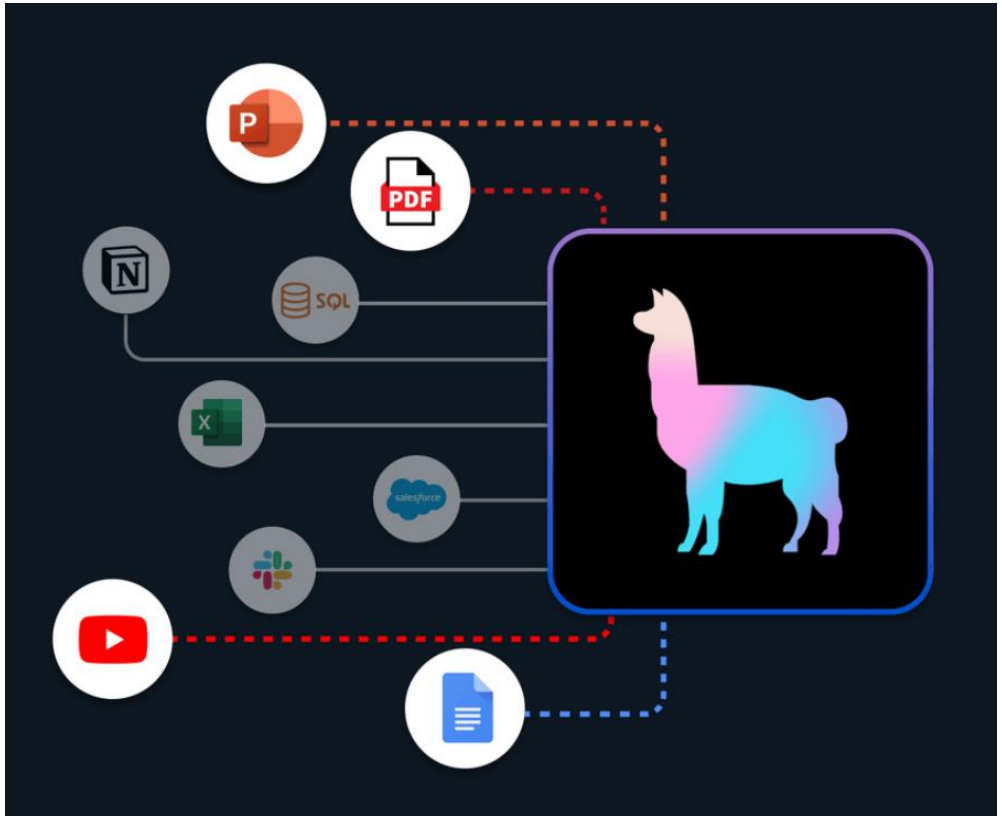
- Llama 2 (https://ai.meta.com/llama/) - open-source large language model (integration with LangChain)
  - SeamlessM4T provides high-quality translation, allowing people from different linguistic communities to communicate effortlessly through speech and text. (August 22, 2023)



Term Project Suggestions using latest technology

Meta's Tools such as:

- FAISS - Efficient similarity search (https://ai.meta.com/tools/faiss/#:~:text=FAISS%20(Facebook%20AI%20Similarity%20Search,more%20scalable%20similarity%20search%20functions)
- Llama 2 (https://ai.meta.com/llama/) - open-source large language model (integration with LangChain)
- LlamaIndex - a simple, flexible data framework for connecting custom data sources to large language models.

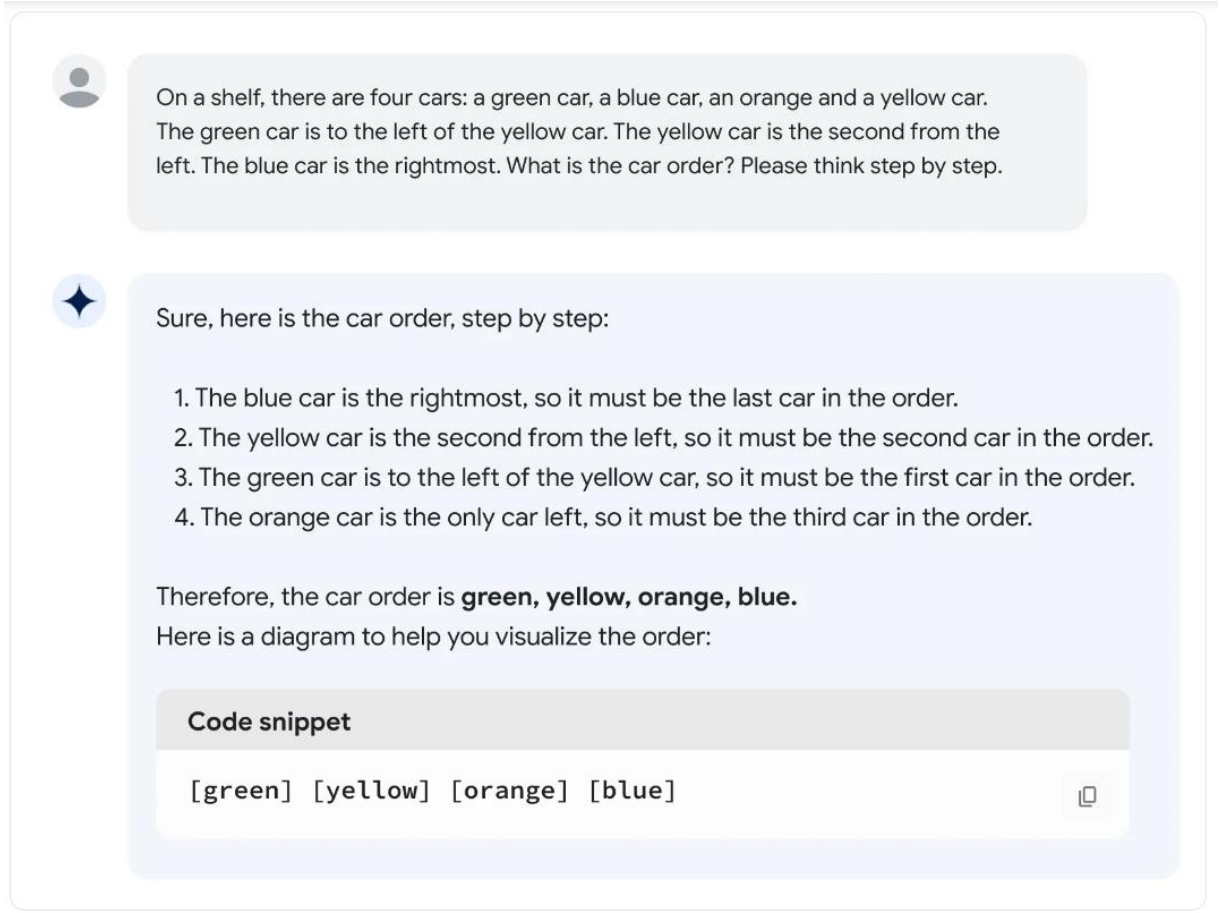


Term Project Suggestions using latest technology

Google's tools such as PaLM 2 – Implement them as Strimlit App (or similar) UI

- An LLM excels at advanced reasoning tasks, including code and math, classification and question answering, translation and multilingual proficiency, and natural language generation better than our previous state-of-the-art LLMs, including PaLM.
  - https://ai.google/discover/palm2/#:~:text=PaLM%202%20is%20our%20next,machine%20learning%20and%20responsible%20AI.

Examples - Reasoning:

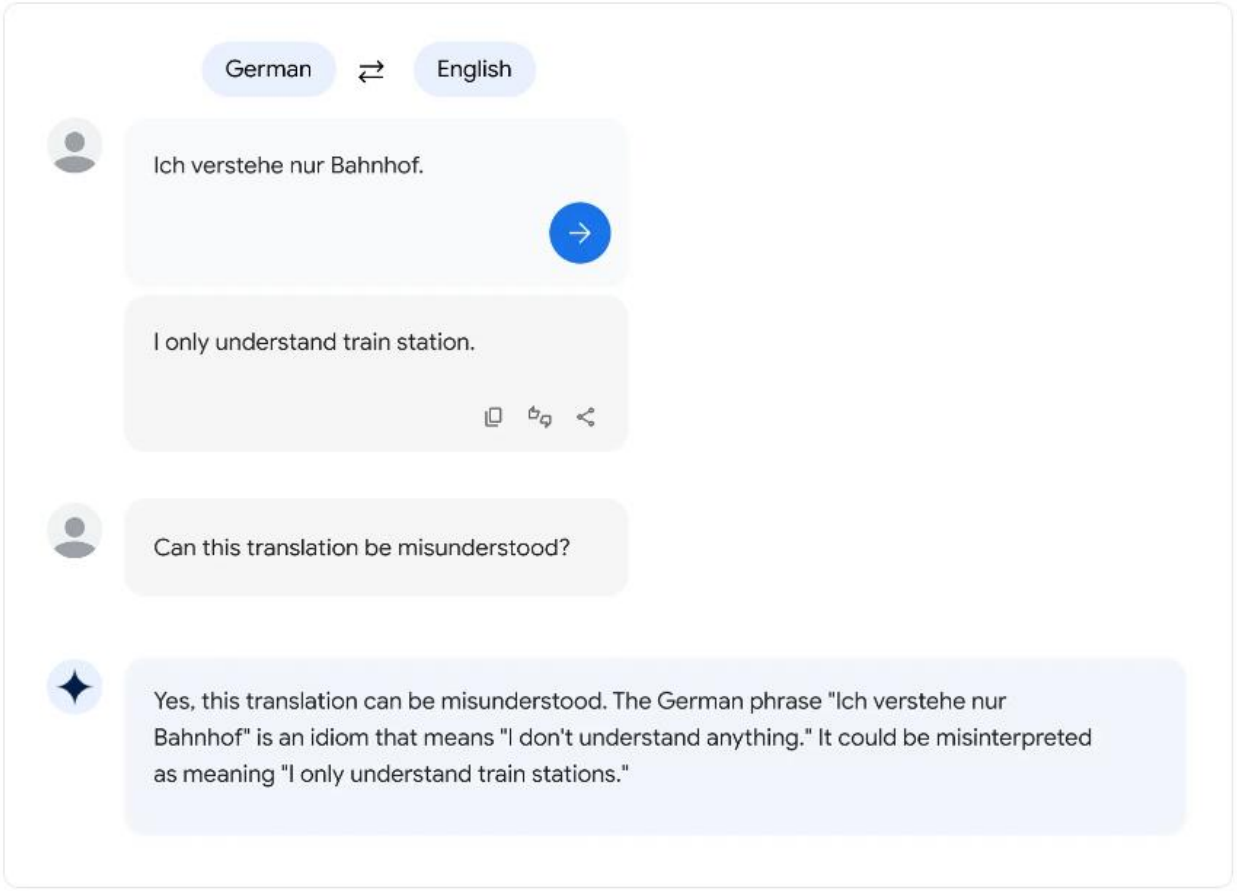


Term Project Suggestions using latest technology

Google's tools such as PaLM 2 – Implement them as Strimlit App (or similar) UI

- An LLM excels at advanced reasoning tasks, including code and math, classification and question answering, translation and multilingual proficiency, and natural language generation better than our previous state-of-the-art LLMs, including PaLM.
  - https://ai.google/discover/palm2/#:~:text=PaLM%202%20is%20our%20next,machine%20learning%20and%20responsible%20AI.

Examples - Multilingual translation:



Term Project Suggestions using latest technology

Google's tools such as PaLM 2 – Implement them as Strimlit App (or similar) UI

- An LLM excels at advanced reasoning tasks, including code and math, classification and question answering, translation and multilingual proficiency, and natural language generation better than our previous state-of-the-art LLMs, including PaLM.
  - https://ai.google/discover/palm2/#:~:text=PaLM%202%20is%20our%20next,machine%20learning%20and%20responsible%20AI.

Examples - Coding:



can you fix this code with a bug and add line by line comments in Korean.

```
visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

The bug in the code is that the `'visited'` set is not being updated. The code should be updated to add the current node to the `'visited'` set before calling the `'dfs'` function recursively. The updated code with Korean comments is as follows:

```
# 방문한 노드의 집합을 만듭니다.
visited = set()

# 깊이 우선 탐색을 수행합니다.
def dfs(visited, graph, node):
    # 현재 노드가 방문한 노드가 아니라면
    if node not in visited:
        # 현재 노드를 방문한 노드로 표시합니다.
        visited.add(node)
        # 현재 노드를 출력합니다.
        print(node)
        # 현재 노드의 인접 노드에 대해 깊이 우선 탐색을 수행합니다.
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)
```

Term Project Suggestions using latest technology

Google's Temporian – open-source Python library for preprocessing and feature engineering temporal data 📈 for machine learning applications.

- https://www.edge-ai-vision.com/2023/09/introducing-temporian-tryolabs-and-google-venture-in-temporal-data-processing/

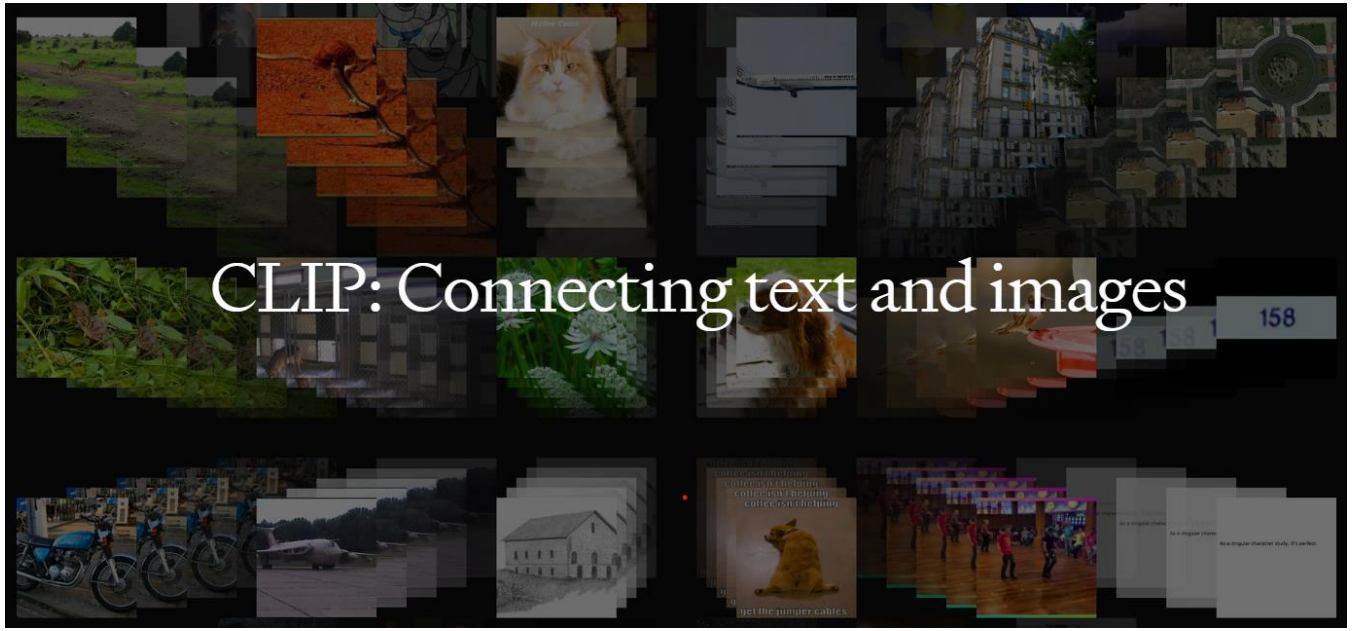
Avoids future leakage:

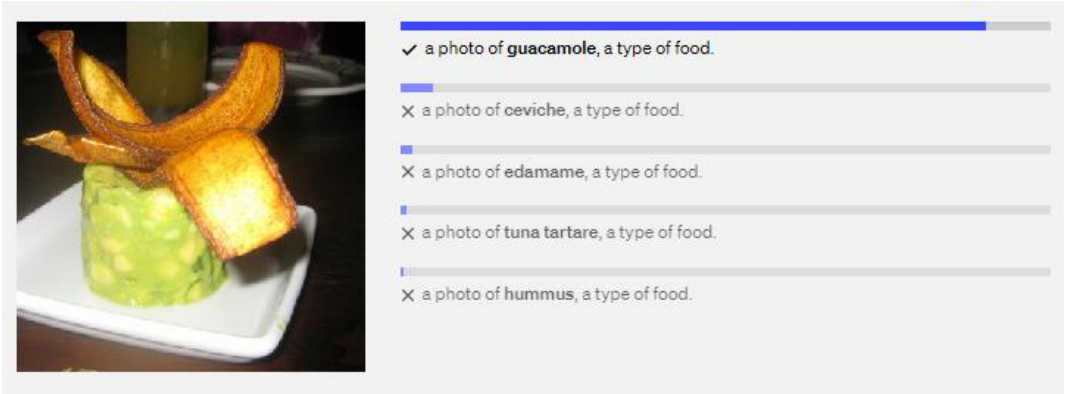
- Future leakage occurs during model training when a model is exposed to data from future events, which leaks information that would otherwise not be available to the model and can result in overfitting.
- Temporian operators do not create leakage by default. Users can also use Temporian to programmatically detect whether specific signals were exposed to future leakages.

Term Project Suggestions using latest technology

Integration of Image Processing with LLMs. Most of these tools were developed in the first half of 2023.

- Visual Bert
- Segment Anything https://segment-anything.com/
- OpenAI CLIP: Connecting text and images https://openai.com/research/clip
- Meta DeepFace (Face Verification) https://research.facebook.com/publications/deepface-closing-the-gap-to-human-level-performance-in-face-verification/





Term Project Suggestions using latest technology

Time-series forecasting with deep learning - Integration of LLM concepts in Finance

- The encoder-decoder models rely on RNN layers. As such, the encoder-decoder concepts has been used for some time already in many of the DL models such as:
  - Convolutional Neural Networks (CNNs)
    - Traditionally designed and used for extracting local relationships in images that are invariant across spatial dimensions.
  - Recurrent Neural Networks (RNNs)
    - Historically they have been used for finding patterns in time-series data since they have an internal memory state which acts as a compact summary of past information.
  - Long Short-Term Memory (LSTM) RNNs
    - They have been developed to address the memory limitations of older variants of RNNs.
- One of the latest ideas is based on adding Attention mechanism to Sqe2Seq models to forecast equity prices.
  - Financial data is not readily available online, so I can provide some samples.
- Relevant papers:
  - Lim, B, Zohren S. 2021 Time-series forecasting with deep learning: a survey. Phil. Trans. R. Soc. A 379: 20200209. https://doi.org/10.1098/rsta.2020.0209
  - https://arxiv.org/abs/2105.10430

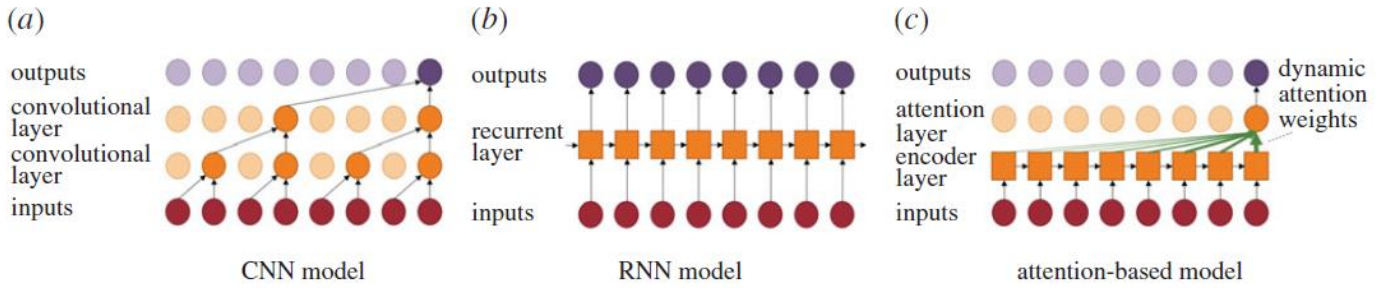


Figure 1. Incorporating temporal information using different encoder architectures. (*a*) CNN model, (*b*) RNN model and (*c*) attention-based model. (Online version in colour.)

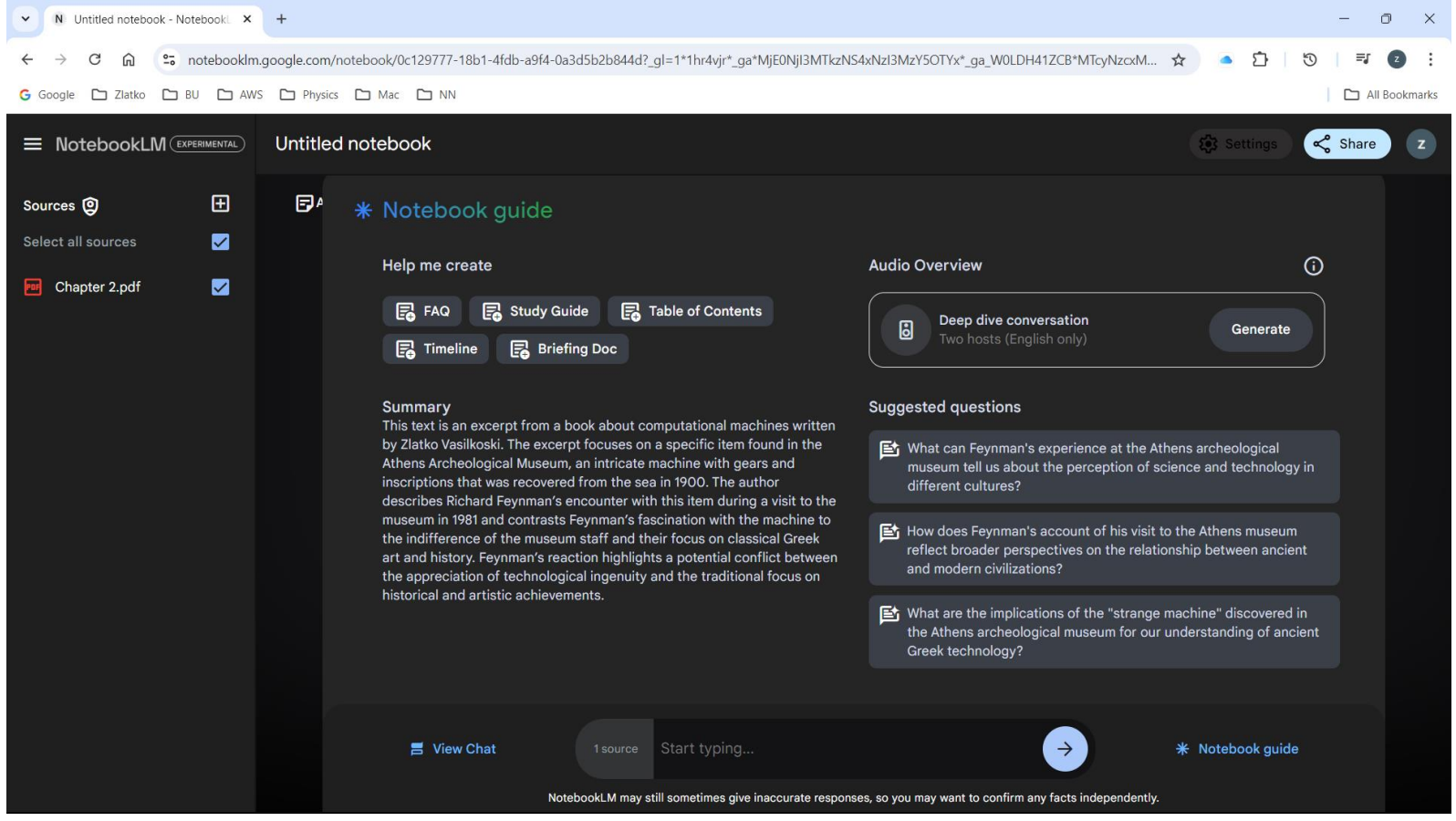
Term Project Suggestions using latest technology

Google NotebookLM is an AI-powered notebook that combines the capabilities of a research assistant with the flexibility of a note-taking tool. It leverages Google's Gemini 1.5 Pro model to provide personalized insights and assistance based on your uploaded documents. Use it in a Streamlit App to create:

- **Podcasts: P**rovide a text file and automatically create a realistic-sounding podcast from it. Integrate it into Streamlit App.
- **Personalized Research Assistant**: By uploading your documents, NotebookLM becomes an expert on your specific topics, allowing it to summarize facts, explain complex ideas, and brainstorm new connections.
- **Source-Grounded Responses**: Provides responses grounded in your sources, citing relevant quotes and information from your uploaded documents.

Easy to use:

- Sign up with a free account
- Click “Try NotebookLM”
- Click on “New Notebook”
- Upload any text or files
- Click “Generate” near the top right in the “Audio Overview” box
- Then you can download this audio file and upload it as a podcast, an audio file, or even make a YouTube video out of it.



Term Project Suggestions using latest technology

A

A

- A